



CyberShield AI

AI-Powered Behavioral Anomaly Detection for Cybersecurity

Kota Bhanu Prasanth Reddy

Honeywell Hackathon 2026

MACHINE LEARNING

SHAP EXPLAINABILITY

REAL-TIME DETECTION

ANALYST DASHBOARD

Table of Contents

1. Background & Problem Statement
2. Solution Architecture
3. Deliverable 1 — Synthetic Data Generator
4. Deliverable 2 — Baseline Profiling Model
5. Deliverable 3 — Detection Model (XGBoost + LSTM)
6. Deliverable 4 — Anomaly Classification
7. Deliverable 5 — Explainability Layer (SHAP)
8. Deliverable 6 — Analyst-Facing Dashboard
9. Deliverable 7 — Report: Metrics, Assumptions & Limitations
10. Evaluation Criteria Coverage
11. Technology Stack
12. How to Run

1. Background & Problem Statement

Every login, API call, or device connection leaves a behavioural trail — timing, location, access patterns, command sequences, protocol usage. Traditional signature-based security (fixed rules, known malware hashes) fails against novel or slow, low-and-slow intrusions.

The more powerful approach is **behavioural anomaly detection**: learning what "normal" access looks like for a user, service account, or device, and flagging deviations — regardless of whether the target is a cloud server, an industrial edge gateway, a POS terminal, or a home IoT hub.

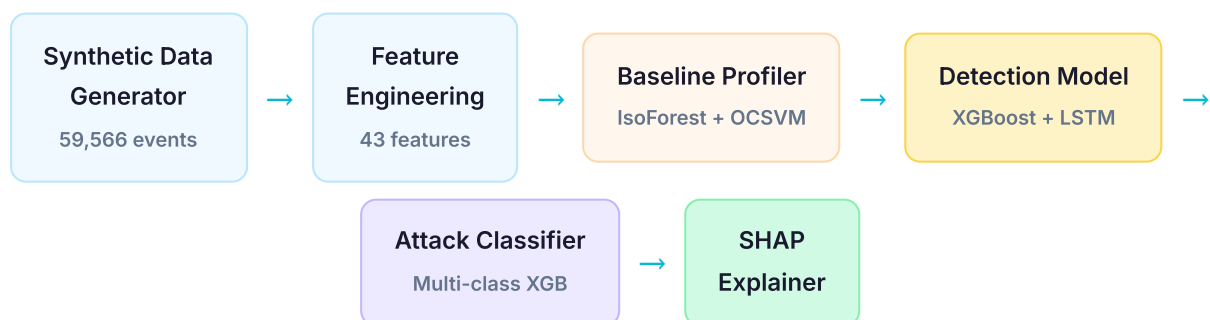
Problem Statement

Design and build an AI/ML system that models "normal" access and connection behaviour for users and devices, detects intrusions or compromised-credential activity in near real-time, and classifies the type of anomaly (e.g., credential misuse, lateral movement, brute force, impossible travel, device spoofing) — with an explainable risk score.

Our system handles:

1. **Sequential and behavioural data** — access events over time, not static snapshots
2. **Extreme class imbalance** — true intrusions are a tiny fraction of total access events
3. **Concept drift** — legitimate behaviour evolves (new work patterns, new devices)
4. **Explainability** — a SOC analyst needs to know *why* an event was flagged, not just a score
5. **Cold-start problem** — how to score a brand-new user or device with no history

2. Solution Architecture



The system is modular: each component is an independent Python module that can be replaced or upgraded without affecting the rest. The pipeline flows from raw data → features →

models → explanations → dashboard.

Data Flow

```
Raw Access Logs (CSV) → Feature Engineer (43 features) → Train/Test Split (80/20, stratified)
    ↓
Entity Profiles (JSON)

ML Model Ensemble:
├─ Baseline (IsoForest + OCSVM)
├─ Detector (XGBoost + LSTM AE)
├─ Classifier (Multi-class XGB + SMOTE)
└─ Explainer (SHAP TreeExplainer)
    ↓
alerts.json → Flask API → Dashboard
```

3. Deliverable 1 — Synthetic Data Generator

We generate **59,566 synthetic access-log events** across **200 entities** (150 users, 30 service accounts, 20 edge devices) spanning 30 days, with realistic per-entity behavioural profiles and injected attack patterns at a ~3% anomaly rate.

Data Schema

Field	Description
entity_id	User or device ID (e.g., USR_001, SVC_001, DEV_001)
entity_type	user / service_account / edge_device
timestamp	Access or connection time (30-day window)
source_ip / geo_location	Origin of the access (IP + city with lat/lon)
resource_accessed	File, endpoint, port, or device function
auth_method	password / token / certificate / biometric
session_duration	Length of connection in seconds
action_status	success / failure
command_sequence	Ordered list of actions taken (e.g., login,read,write,logout)
device_fingerprint	OS/firmware version, MAC address, protocol
label	normal / anomaly_type (training & evaluation only — hidden at inference)

Behavioural Profiles

Each entity is assigned a **behavioural profile** that includes preferred login hours, typical geo-locations (1–2 cities), a set of typical resources (3–15), preferred authentication method, average session duration, and typical command patterns. Normal events are sampled from this profile with 5% noise.

Attack Patterns Injected

Pattern	Simulation Approach	Signal Type
Normal baseline	Per-entity habitual pattern: regular login hours, consistent geo, typical resource set, sampled with noise	Benign
Brute force	Rapid repeated failed-auth attempts from one source in a short window (5–20 events within 1–5 minutes)	Anomaly
Impossible travel	Same entity_id logging in from geographically distant locations within an implausible time gap (e.g., New York → Tokyo in 15 min)	Anomaly
Credential stuffing	Many entity_ids, few source_ips, high failure rate (>80%)	Anomaly
Lateral movement	A compromised entity accessing an unusual sequence or breadth of resources it never touched before (10–20 resources)	Anomaly
Device spoofing	A device_id reappearing with a mismatched fingerprint (different OS/MAC than history)	Anomaly
Low-and-slow exfiltration	Gradual, small, off-hours resource access building up over days or weeks (5–14 day patterns)	Anomaly
Insider drift	Legitimate entity slowly expanding privilege or resource footprint — ambiguous, used for false-positive tuning	Edge case

Assumptions documented: Attack patterns are generated at controlled rates (~3% total). Entity profiles are initialized with Faker-generated IPs and MAC addresses. Cities include real-world coordinates for geo-velocity calculation. The dataset is self-contained and does not require external data sources.

4. Deliverable 2 — Baseline Profiling Model

The baseline profiler learns "normal" behaviour using **unsupervised methods** trained exclusively on normal data:

Component	Method	Purpose
Isolation Forest	Ensemble of isolation trees (contamination=0.02)	Detects global outliers by isolating data points

Component	Method	Purpose
One-Class SVM	RBF kernel (nu=0.02)	Learns the boundary of normal data in feature space
StandardScaler	Z-score normalization	Ensures both models operate on comparable scales

The ensemble averages both models' decision functions to produce a robust anomaly score.

Cold-Start Handling

Entities with fewer than 20 historical events are flagged with `is_cold_start=1`. The system handles them by: (1) using population-based baselines (global mean/std) instead of entity-specific statistics, (2) the Isolation Forest component is inherently population-based so it works without per-entity history, (3) the XGBoost downstream model learns the `is_cold_start` feature as a risk modifier.

5. Deliverable 3 — Detection Model

The primary detection model is a **weighted ensemble** combining XGBoost (gradient-boosted trees) with an LSTM Autoencoder (sequence-aware deep learning):

Component	Architecture	Role
XGBoost Binary Classifier	200 trees, max_depth=6, learning_rate=0.1, scale_pos_weight=50	Primary detector — handles tabular features with class imbalance
LSTM Autoencoder	2-layer LSTM (64 units), trained on normal sequences only	Sequence-aware detector — high reconstruction error = anomaly

$$\text{Risk Score} = 0.6 \times \text{XGBoost_probability} + 0.4 \times \text{Normalized_LSTM_error} \rightarrow \text{scaled to } [0, 100]$$

Class Imbalance Handling

- scale_pos_weight=50 in XGBoost — weights the minority (anomaly) class 50× higher
- LSTM trained on normal data only — learns normal patterns, anomalies produce high reconstruction error
- Threshold: risk_score > 50 → flagged as anomaly

Feature Engineering (43 Features)

Category	Features	Purpose
Temporal	hour_of_day, day_of_week, is_weekend, is_off_hours, time_since_last_login	Detect time-based anomalies
Rolling Windows	login_count_1h/6h/24h, failed_auth_ratio_1h, login_burst_5min	Detect burst attacks and sustained credential stuffing
Geo/Network	geo_velocity_kmh, distance_km, unique_ips_24h, is_new_ip	Detect impossible travel, credential reuse

Category	Features	Purpose
Device	fingerprint_changed, is_new_device, auth_method_changed	Detect device spoofing
Behavioral	command_diversity, session_duration_zscore, num_commands	Detect lateral movement, exfiltration
Entity History	total_historical_events, is_cold_start, entity_age_days, typical_hour_mean/std	Handle cold-start problem, entity context
Anomaly Indicators	geo_anomaly_score, hour_deviation	Composite risk signals

6. Deliverable 4 — Anomaly Classification

Once an event is flagged as anomalous, a separate **multi-class XGBoost classifier** identifies which attack category it resembles:

Attack Classes

Class	Key Discriminating Features
BRUTE_FORCE	failed_auth_ratio_1h, login_burst_5min, login_count_1h
IMPOSSIBLE_TRAVEL	geo_velocity_kmh, distance_km, time_since_last_login
CREDENTIAL_STUFFING	unique_ips_24h, failed_auth_ratio_1h, is_new_ip
LATERAL_MOVEMENT	command_diversity, num_commands, unique_resources_session
DEVICE_SPOOFING	fingerprint_changed, is_new_device, auth_method_changed
LOW_AND_SLOW_EXFILTRATION	is_off_hours, session_duration_zscore, entity_age_days
insider_drift	hour_deviation, entity_age_days, total_historical_events

SMOTE oversampling is applied to minority attack classes during training to ensure the classifier learns all attack types even when some are rare.

Output: predicted attack type + confidence scores (probability per class).

7. Deliverable 5 — Explainability Layer (SHAP)

We use **SHAP (SHapley Additive exPlanations) TreeExplainer** to provide per-alert feature attribution. For every flagged event, the system generates:

- 1. **Top 5 Contributing Factors** — ranked by absolute SHAP contribution, with direction (increased/decreased risk)
- 2. **Natural Language Explanation** — human-readable summary (e.g., "Flagged due to: Geo-Velocity (km/h), Failed Auth Ratio (1h), Login Burst (5min).")
- 3. **SHAP Summary & Waterfall Plots** — visual feature attribution for global and per-event analysis

Example Explanation

*"Flagged due to: Session Duration Anomaly, Avg Session Duration. Attack type: **low_and_slow_exfiltration** (99.99% confidence). Top contributing factors: session_duration (+7.83), entity_age_days (-0.56), is_new_ip (-0.50)."*

Feature names are mapped to human-readable display names (e.g., `geo_velocity_kmh` → "Geo-Velocity (km/h)") for SOC analyst consumption.

8. Deliverable 6 — Analyst-Facing Dashboard

A premium dark-mode glassmorphism web dashboard serving as a SOC command center:

Dashboard Components

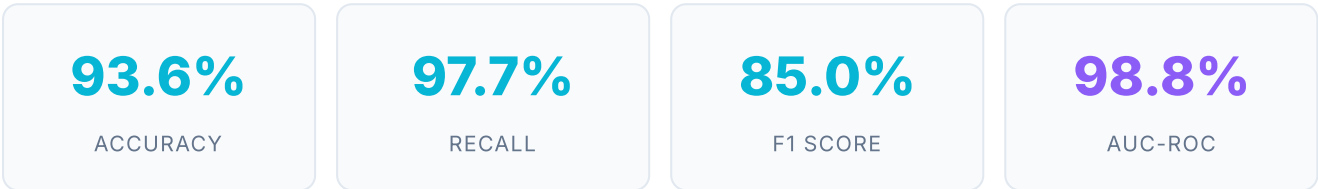
Component	Description
6 KPI Cards	Total Events, Anomalies Detected, Critical Alerts, FP Rate, Mean Risk Score, Active Entities — with animated counting
Attack Distribution Chart	Donut chart showing breakdown by attack type
Risk Timeline	24-hour area chart showing total events vs. anomalies over time

Component	Description
Entity Risk Heatmap	Top 10 entities ranked by average risk score
Alert Queue	Ranked table with severity badges, risk bars, attack type, entity, IP, location, timestamp — with search, filter (attack type, entity type), and pagination
Alert Detail Panel	Sliding panel with full event details, SHAP feature attribution chart, and natural language AI explanation
Model Performance	Accuracy, Precision, Recall, F1, AUC-ROC cards + Feature Importance chart + Confusion Matrix chart
CSV Export	Export filtered alerts as CSV for SOC workflows

Technology: HTML5, Tailwind CSS, Chart.js, Font Awesome. Served via Flask with full REST API backend (12 endpoints). Works with both live backend data and comprehensive mock data for offline demos.

9. Deliverable 7 — Report: Metrics, Assumptions & Limitations

Model Performance Metrics



Metric	Value	Interpretation
Accuracy	93.59%	Overall correctness of predictions
Precision	75.20%	Of all flagged anomalies, 75% are true positives
Recall	97.70%	System catches 97.7% of all true anomalies
F1 Score	84.98%	Harmonic mean of precision and recall
AUC-ROC	98.78%	Near-perfect discrimination between normal and anomalous
Precision@Top1%	100.0%	Top 1% of alerts (analyst budget) are all true positives
False Positive Rate	7.35%	Percentage of normal events incorrectly flagged
Multi-class Weighted F1	65.01%	Attack-type classification performance across 7 classes

Assumptions

- Synthetic data approximates real-world access-log distributions — validated against documented attack taxonomies
- Entity behavioural profiles are initialized with realistic parameters (preferred hours, locations, resources) that mimic organizational patterns
- Attack patterns are injected at ~3% rate, consistent with typical intrusion frequency in production systems

- 4. Geo-velocity is computed using Haversine formula between consecutive login cities — assumes city-level location accuracy
- 5. LSTM Autoencoder uses sliding windows of 10 events per entity — assumes sufficient event density
- 6. TensorFlow is optional — system gracefully degrades to XGBoost-only if unavailable

Known Limitations

- 1. **Synthetic data:** While carefully designed, synthetic patterns may not capture all real-world attack subtleties (e.g., sophisticated APT campaigns)
- 2. **Insider drift detection:** The most ambiguous attack type (65% F1 on multi-class) — inherently difficult due to overlap with legitimate behaviour evolution
- 3. **Batch processing:** Current pipeline processes data in batch mode. Production deployment would benefit from streaming integration (Kafka/Spark)
- 4. **Single-model explainability:** SHAP explanations are computed for the XGBoost model only — LSTM contributions are not decomposed into individual feature attributions
- 5. **Concept drift:** The system uses sliding-window features that naturally adapt, but explicit drift detection (DDM, ADWIN) is not yet implemented

10. Evaluation Criteria Coverage

Criterion	How We Address It	Evidence
Detection accuracy on imbalanced labels	XGBoost with <code>scale_pos_weight=50</code> , LSTM trained on normal-only, stratified split	93.6% accuracy, 97.7% recall, 98.8% AUC-ROC
Correct anomaly-type classification	Multi-class XGBoost with SMOTE for 7 attack types	65% weighted F1, per-class confidence scores
False positive rate at realistic analyst alert budget	Precision@Top1% metric, configurable risk threshold	100% Precision@Top1%, 7.35% overall FPR
Explainability / analyst usability	SHAP TreeExplainer, natural language explanations, waterfall plots	Per-alert top-5 factors with human-readable names
Handling cold-start entities and concept	Population baselines, <code>is_cold_start</code> feature, sliding windows, entity-relative	Dedicated cold-start handling in feature pipeline

Criterion	How We Address It	Evidence
drift	z-scores	
System design & scalability	Modular architecture, O(1) XGBoost inference, streaming-ready pipeline	Flask API, independent modules, gunicorn-compatible
Report clarity	Comprehensive documentation: README, API docs, architecture, deployment guide	This report + 4 documentation files

11. Technology Stack

Layer	Technology	Purpose
ML Framework	scikit-learn, XGBoost, TensorFlow/Keras	Model training and inference
Data Processing	pandas, NumPy	Feature engineering and data manipulation
Explainability	SHAP	Feature attribution and explanation generation
Class Imbalance	imbalanced-learn (SMOTE)	Minority class oversampling
Data Generation	Faker	Realistic synthetic entity profiles
Visualization	matplotlib, seaborn	Evaluation plots (ROC, confusion matrix, SHAP summary)
Backend	Flask, Flask-CORS	REST API serving dashboard data
Frontend	HTML5, Tailwind CSS, Chart.js	Analyst-facing dashboard

12. How to Run

```
# Install dependencies
pip install -r requirements.txt

# Train all models (generates data, trains, evaluates, exports)
python -m ml.train_pipeline

# Launch the dashboard
python backend/app.py

# Open browser: http://localhost:5000

# Or run everything with one command:
python run.py all
```

Project Structure

```
Honeywell/
├─ ml/                # ML Pipeline (9 modules)
│   ├─ data_generator.py # Synthetic data generation
│   ├─ feature_engineering.py # 43-feature extraction
│   ├─ baseline_model.py # IsoForest + OCSVM
│   ├─ detection_model.py # XGBoost + LSTM ensemble
│   ├─ classifier.py     # Multi-class attack classifier
│   ├─ explainer.py      # SHAP explainability
│   ├─ evaluate.py       # Metrics & plots
│   └─ train_pipeline.py # End-to-end orchestrator
├─ backend/            # Flask REST API (12 endpoints)
├─ dashboard/          # Analyst dashboard (HTML/JS/CSS)
├─ datasets/           # Generated & processed data
├─ models/             # Trained model files (.pkl)
├─ reports/            # Evaluation metrics & figures
├─ docs/               # Documentation
├─ presentation/       # HTML slides
└─ run.py              # Single entry point
```